

T-Optimal Synthesis of 2 Qubit Clifford + T Circuits Using Shortest Path Graph Searches and a Barnes-Wall Lattice Basis

Katie Keiller

Supervisors: Julien Ross, Alexandre Clément

September 12, 2026

Abstract

In this report, recent exact circuit synthesis algorithms using an A* graph search by V. Kliuchnikov and S. Schönnenbeck are explicitly demonstrated on 1- and 2-qubit Clifford + T circuits. These algorithms minimise the use of T-gates which is desirable for fault-tolerant computing. Edges, vertices and heuristic functions are defined by computing Hermite and Smith normal forms of matrices over the ring $\mathbb{Z}[e^{i\pi/4}]$. A Python GitHub repository containing implementations is provided.

Acknowledgements

This is a report of a summer research project completed at Dalhousie University through the Mitacs Globalinks programme. I would like to thank Mitacs for generously funding this research project and for giving me the opportunity to work in Canada. I am also immensely grateful to my supervisors for guiding me during this internship and for introducing me to their field of research.

Contents

1	Introduction	2
2	Preliminaries	2
2.1	Rings	2
2.2	Clifford + T Gate Set	3
2.3	Barnes-Wall Basis	3
2.4	Matrix Normal Forms	4
2.4.1	Hermite Normal Form	4
2.4.2	Smith Normal Form	5
3	Defining the Graph Search	6
3.1	Representation of U	6
3.2	Minimal Generator Sets	6
3.3	Vertices and Edges	7
4	A* Search	7
4.1	Heuristics	7
4.1.1	LDE Heuristic	8
4.1.2	$\bar{k}$ -Heuristic	8
4.2	Algorithm	10
4.3	Demonstration	11
5	Matsumoto-Amano Form Search	11
5.1	Algorithm	11
5.2	Demonstration	12
6	Concluding Remarks	12

7	Appendix	13
7.1	Minimal Generating Sets	13
7.2	Algorithms for Matrix Normal Forms	13
7.3	Python Implementations	15
8	References	15

1 Introduction

Quantum circuits acting on n qubits can be represented as $2^n \times 2^n$ unitary matrices built from the tensor product and composition of matrices representing an appropriate gate set. Whilst calculating the matrix representation of a circuit is straightforward, finding a sequence of tensor products and compositions that exactly builds a given circuit unitary is a much more complex problem known as the exact synthesis problem. This can be thought of as a matrix analogue of factorisation. It has been shown that exact circuit synthesis is possible provided the gate set is appropriately chosen. The gate set required is dependent on the ring that the entries of the circuit unitary belong to, and in this report we will consider circuits represented by unitary matrices with entries in $\mathbb{Z}[\omega, 1/2]$ where ω is $e^{i\pi/4}$.

The Clifford + T gate set is *universal*, meaning that it can approximately synthesise any unitary matrix to an arbitrary accuracy. The entries of these gates also belong to the ring $\mathbb{Z}[\omega, 1/2]$ and as such it can be used to *exactly* synthesise a group of these unitaries. There are infinite possible ways to synthesise such circuits, and this raises the question of optimisation. Naturally, we may consider the synthesis that minimises the total number of gates to be the "best". However in the context of fault-tolerant computing, some gates are more "costly" to implement than others. We therefore consider an optimal synthesis to be one that minimises the use of this "costly" subset of gates.

In Section 2, we first define useful properties of the relevant rings, $\mathbb{Z}[\omega]$ and $\mathbb{Z}[\omega, 1/2]$, and the Clifford + T gate set. We then describe how a basis change into the Barnes-Wall basis allows us to consider the cost-free subset of the Clifford + T gate set as integer matrices over $\mathbb{Z}[\omega]$ whilst the costly subset remains matrices over $\mathbb{Z}[\omega, 1/2]$. The synthesis algorithms are based on the idea that, in the Barnes-Wall basis, an algorithm to reduce a unitary U over $\mathbb{Z}[\omega, 1/2]$ to a matrix over $\mathbb{Z}[\omega]$, is the inverse of an algorithm to synthesise U . We then describe two normal forms of matrices: Hermite and Smith. The former is used in Section 3 to define equivalence classes of unitaries which represent vertices in a graph, and the latter is used in Section 4.1.2 to define an A* heuristic function. Algorithms relating to the computation of matrix normal forms are left to the appendix.

In Section 3, we transform the exact circuit synthesis problem into a shortest-path graph search by considering the number of T-gates used in a synthesis to be the "length" of a path. It is useful to consider the circuit unitary U as a sequence of cost-free gates separated by a minimal amount of costly gates, and by doing so we can define U as a sequence of generators. The algorithm to construct this set of generators uses the Hermite normal form from Section 2, and the lists of these generators is given in the appendix.

In Section 4 we first define an A* search and two consistent heuristic functions for the graph defined in Section 3. We then see a demonstration of the algorithm for a 2-qubit Clifford + T circuit. In Section 5, we briefly describe the Matsumoto-Amano canonical form of 1-qubit Clifford + T circuits and see that the A* graph search can be used to find this canonical form.

Section 6 summarises how the algorithm may be adapted and includes a brief description of calculating a basic lower bound on the number of T-gates required to synthesise a given circuit unitary. A Python GitHub repository is provided in the appendix; this contains implementations of every algorithm described in the report, as well as the option to synthesise unitaries over different rings which require other, related, gate sets.

In this report, proofs are provided for lemmas and corollaries, however theorems are stated without proof.

2 Preliminaries

2.1 Rings

There are two related rings which are used in this report. All matrices considered will have their entries in one of these rings.

Definition 2.1. *The ring $\mathbb{Z}[\omega]$:*

$$\mathbb{Z}[\omega] = \{a + b\omega + c\omega^2 + d\omega^3 : a, b, c, d \in \mathbb{Z}, \omega = e^{i\pi/4}\} \quad (1)$$

Definition 2.2 (Equation 2 of [5]). *The **norm** $N(z)$ on $\mathbb{Z}[\omega]$ is a multiplicative function $N : \mathbb{Z}[\omega] \rightarrow \mathbb{N}$:*

$$N(z) = \prod_{k \in \{1,3,5,7\}} (a + b\omega^k + c\omega^{2k} + d\omega^{3k}) \quad (2)$$

Examples of units in $\mathbb{Z}[\omega]$ are ω and $\sqrt{2} \pm 1$.

Theorem 2.3 (Theorem 1 of [12]). $\mathbb{Z}[\omega]$ is a Euclidean Domain.

Definition 2.4. The ring $\mathbb{Z}[\omega, 1/2]$:

$$\mathbb{Z}[\omega, 1/2] = \left\{ \frac{a}{\delta^k} : a \in \mathbb{Z}[\omega], \delta = (1 + \omega), k \in \mathbb{Z} \right\} \quad (3)$$

This representation is justified since $1/2 = (1 + \omega)^{-4}$ up to units. From here onwards, δ is taken to be $1 + \omega$.

Lemma 2.5. δ is prime in $\mathbb{Z}[\omega]$

Proof. $N(\delta) = 2$. Since the norm is a multiplicative function into $\mathbb{N}$, this means that if $vu = \delta$, then u, v must have norms 1, 2 so one of u, v is a unit and δ is irreducible. Since $\mathbb{Z}[\omega]$ is a Euclidean Domain, this means δ is prime. $\square$

2.2 Clifford + T Gate Set

Gate sets are a set of "simpler" matrices which can be composed and tensored together to synthesise circuits. A common gate set used to synthesise unitaries over $\mathbb{Z}[\omega, 1/2]$ is the Clifford + T gate set.

A $2^n \times 2^n$ unitary represents a transformation on n qubits; in this report we define $N = 2^n$ and focus on the cases where $n = 1$ or 2 .

- For single qubit circuits, the Clifford + T set can exactly synthesise a unitary, U , without the use of an ancilla, if and only if $\det(U) = \omega^k$ for $k \in \{0, 1, 2, 3, 4, 5, 6, 7\}$. (Corollary 2.b of [10])
- For two qubit circuits with this gate set, ancilla-free synthesis is restricted to unitaries with determinant equal to ω^{2k} for $k \in \{0, 1, 2, 3\}$. (Corollary 2.b of [10])

We use a globally phase-adjusted Hadamard gate (since it simplifies operations in this ring and does not have an effect on the T-optimal synthesis) $\tilde{H}$, and the Clifford + T gates are as follows:

$$\tilde{H} = \frac{1}{1+i} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \quad CX = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad T = \begin{pmatrix} 1 & 0 \\ 0 & \omega \end{pmatrix} \quad (4)$$

- The **1-qubit** Clifford + T set is $\{\tilde{H}, S, T\}$.
- The **n-qubit** Clifford + T set is $\{\tilde{H}, S, CX, T\}$ where it is assumed each gate is tensored with an appropriately sized identity matrix to become a $2^n \times 2^n$ matrix, and that all combinations of control- and target- qubits are included.

Definition 2.6. The cost-free subset, C , of the Clifford + T gate set is $\{\tilde{H}, S, CX\}$. We call matrices synthesised from only these gates **Cliffords**.

Definition 2.7. The costly subset, G , of the Clifford + T gate set is $\{T\}$.

We now define an optimisation problem in the context of fault-tolerant exact circuit synthesis using the Clifford + T gate set, which is the main subject of this report.

Problem 1 (T-Optimal Exact Synthesis). Given a $2^n \times 2^n$ unitary matrix U representing an n -qubit quantum circuit with entries in $\mathbb{Z}[\omega, 1/2]$, synthesise U using the fewest T gates as possible.

2.3 Barnes-Wall Basis

The Barnes-Wall basis is integral to the algorithms described in Sections 3, 4, and 5. It allows us to consider the set C as having entries in $\mathbb{Z}[\omega]$ and the set G as having entries in $\mathbb{Z}[\omega, 1/2]$.

Definition 2.8 (Equation 3 of [2]). Over $\mathbb{Z}[\omega, 1/2]$ the Barnes-Wall basis matrices are defined as

$$B = \begin{pmatrix} \frac{1}{1+i} & 0 \\ \frac{1}{1+i} & 1 \end{pmatrix} \quad B_N = B^{\otimes n} \text{ is the } 2^n \times 2^n \text{ Barnes-Wall basis matrix.} \quad (5)$$

For any $N \times N$ matrix A , we define $A_B = B_N^{-1} A B_N$ as the matrix A in the Barnes-Wall basis. For example,

$$\tilde{H}_B = B^{-1} \tilde{H} B = \begin{pmatrix} 1+i & i \\ -1 & -1-i \end{pmatrix} \quad T_B = B^{-1} T B = \frac{1}{\delta} \begin{pmatrix} 1+\omega & 0 \\ i & \omega+i \end{pmatrix}$$

Definition 2.9. A matrix is $\mathbb{Z}[\omega]$ -**unimodular** if its entries are elements of $\mathbb{Z}[\omega]$ and the determinant is a unit in $\mathbb{Z}[\omega]$. The inverse of the matrix is also $\mathbb{Z}[\omega]$ -unimodular.

Theorem 2.10. (Kliuchnikov & Schönnenbeck [3](Equation 6 of [2])). A unitary matrix over $\mathbb{Z}[\omega, 1/2]$ transformed into the Barnes-Wall basis is $\mathbb{Z}[\omega]$ -unimodular if and only if it is a Clifford matrix.

The set G also has the trivial property that for any $g \in G$, there exists a matrix A over $\mathbb{Z}[\omega]$ such that $g_B = \delta^{-k} A$, and $k \in \mathbb{N}$ is the greatest **least denominator exponent** of the entries in g_B . As a result, if we multiply g_B by δ^k , we obtain a matrix which is integer over $\mathbb{Z}[\omega]$ but not $\mathbb{Z}[\omega]$ -unimodular.

Definition 2.11 (δ -modular). A matrix M with entries in $\mathbb{Z}[\omega, 1/2]$ is δ -modular if there exists a $\mathbb{Z}[\omega]$ -unimodular matrix, A and $k \in \mathbb{N}$ such that $M = \delta^{-k} A$.

It is useful to note that the result of the scalar multiplication of a $\mathbb{Z}[\omega]$ -unimodular matrix by a unit of $\mathbb{Z}[\omega]$ is also a $\mathbb{Z}[\omega]$ -unimodular matrix.

Lemma 2.12. $B_N^\dagger B_N$ is a δ -modular matrix.

Proof. We show the base case is true: $B^\dagger B = \begin{pmatrix} \frac{1+i}{2} & \frac{1+i}{2} \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \frac{1}{1+i} & 0 \\ \frac{1}{1+i} & 1 \end{pmatrix} = \begin{pmatrix} 1 & \frac{1+i}{2} \\ \frac{1}{1+i} & 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} \omega - \omega^3 & \omega \\ -\omega^3 & \omega - \omega^3 \end{pmatrix} = \frac{1}{\sqrt{2}} M$

M is a $\mathbb{Z}[\omega]$ -unimodular matrix because it is integer over $\mathbb{Z}[\omega]$ and $\text{Det}(M) = i$. Furthermore, $\frac{1}{\sqrt{2}} = u\delta^{-2}$ where u is a unit, so this satisfies the condition that $B_N^\dagger B_N = \delta^{-k} M'$ where M' is a $\mathbb{Z}[\omega]$ -unimodular matrix.

Using results on the tensor product from Section 1 of [7], we show $B_N^\dagger B_N$ is a δ -modular matrix:

$B_N^\dagger B_N = (B^{\otimes n})^\dagger B^{\otimes n} = (B^\dagger B)^{\otimes n} = ((\delta^2 u)^{-1} M)^{\otimes n} = (\delta^2 u)^{-n} M^{\otimes n}$. That $M^{\otimes n}$ is $\mathbb{Z}[\omega]$ -unimodular follows from properties of the determinant of a tensor product, and that if all elements of M are integer, all elements of $M^{\otimes n}$ must also be integer. $\square$

2.4 Matrix Normal Forms

The following matrix normal forms take advantage of the Barnes-Wall basis. They define canonical forms for matrices over $\mathbb{Z}[\omega, 1/2]$ (corresponding to the set G in the Barnes-Wall basis) by transforming them by $\mathbb{Z}[\omega]$ -unimodular matrices (corresponding to matrices from C). The Hermite Normal Form is used primarily in Algorithm 1 and the Smith Normal Form is used in Section 4.1.2.

2.4.1 Hermite Normal Form

A Hermite Normal Form of a full-rank square matrix with entries in a $\mathbb{Z}[\omega]$ is an upper-triangular matrix over $\mathbb{Z}[\omega]$ where the **pivot** elements are on the diagonal and in each column every element above the diagonal is smaller in norm than the pivot. A matrix over $\mathbb{Z}[\omega]$ can be transformed into a Hermite Normal Form by pre-multiplication by a $\mathbb{Z}[\omega]$ -unimodular matrix. See Algorithm 4 in the appendix for an explicit algorithm.

Theorem 2.13 (Proposition 1.2 of [4]). Let A be an $N \times N$ matrix over $\mathbb{Z}[\omega]$, then there exists a $\mathbb{Z}[\omega]$ -unimodular matrix M such that $A = MH$ and H is a Hermite Normal Form of A .

By this definition alone, the Hermite Normal Form of a matrix is not unique. Consider as an example the matrices A, Z, H :

$$A = \begin{pmatrix} 1 & 2 \\ 0 & -3\omega \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 0 & 3\omega \end{pmatrix} = ZH \quad (6)$$

A and H are both valid Hermite Normal Forms of A , and they differ only by pre-multiplication by a $\mathbb{Z}[\omega]$ -unimodular matrix Z .

Definition 2.14. Let R be a ring, we define a canonical element to be an element of R which represents an equivalence class in R where members are associates. That is to say, for any elements a, b in a given class, $a = ub$ where u is a unit in R .

Theorem 2.15 (Definition 2.6 of [4]). Any matrix A over $\mathbb{Z}[\omega]$ has a Unique Hermite Normal Form (UHNF) where the pivots are canonical elements of $\mathbb{Z}[\omega]$, and the elements, x_{ij} above the pivots are unique representatives of the residue of x_{ij} modulo the pivot.

In several algorithms used in the graph search it will be necessary to check if two matrices have the same UHNF. Finding canonical elements is tricky but we can check if two matrices would have the same computed UHNF by using the following properties of Unique Hermite Normal Forms:

Lemma 2.16. Two matrices A, B with entries in $\mathbb{Z}[\omega]$ have the same UHNF if and only if $A = MB$ where M is a $\mathbb{Z}[\omega]$ -unimodular matrix.

Proof.

$\Leftarrow$ Let $A = MB$ and $A = M_A H$ such that M_A, M are $\mathbb{Z}[\omega]$ -unimodular, and H is the UHNF of A . Then $B = M^{-1} M_A H = M_B H$ and H satisfies the conditions of Lemma 2.15, since by assumption it is the UHNF of A . By uniqueness of the UHNF, this means H is also the UHNF of B .

$\Rightarrow$ If A and B have the same UHNF, H then there exist $\mathbb{Z}[\omega]$ -unimodular matrices M_A, M_B such that $A = M_A H$ and $B = M_B H$. Since $\mathbb{Z}[\omega]$ -unimodular matrices are invertible, and their inverses are also $\mathbb{Z}[\omega]$ -unimodular, this means that $M_A^{-1} A = M_B^{-1} B \Leftrightarrow A = M' B$, where $M' = M_A M_B^{-1}$. $\square$

An algorithm to check this condition is Algorithm 6 in the Appendix.

The Hermite Normal Form can be extended to matrices over $\mathbb{Z}[\omega, 1/2]$ using the fact that for every matrix B over $\mathbb{Z}[\omega, 1/2]$, there exists a matrix A over $\mathbb{Z}[\omega]$ such that $B = \delta^{-k} A$ for k the greatest least denominator exponent of B . We can compute the Hermite Normal Form of A and then multiply by δ^{-k} .

Lemma 2.17 (Generalisation to $\mathbb{Z}[\omega, 1/2]$). *Two matrices A, B over $\mathbb{Z}[\omega, 1/2]$ have the same UHNF if and only if $A = MB$ where M is a $\mathbb{Z}[\omega]$ -unimodular matrix.*

Proof.

$\Rightarrow$ Let $A = \delta^{-k} \bar{A}$ and $B = \delta^{-k'} \bar{B}$ such that k, k' are the least denominator exponents of A and B , and $\bar{A}, \bar{B}$ are $\mathbb{Z}[\omega]$ matrices. Assume further that $A = MB$ for some $\mathbb{Z}[\omega]$ -unimodular matrix M . Then $\delta^{k-k'} \bar{A} = M \bar{B}$. The matrices on the right-hand side are both integer over $\mathbb{Z}[\omega]$ and so, by assumption that k be the least denominator exponent of A , we get that $k \geq k'$. Recalling that the inverse of a $\mathbb{Z}[\omega]$ -unimodular matrix is also $\mathbb{Z}[\omega]$ -unimodular, we have from $\delta^{k'-k} \bar{B} = M^{-1} \bar{A}$ that $k' \geq k$, and therefore that $k = k'$. Thus $\bar{A} = M \bar{B}$, and by Lemma 2.16 the UHNFs of $\bar{A}$ and $\bar{B}$ are the same. As we compute the UHNFs of A and B by then multiplying by δ^{-k} , the UHNFs of A and B are also the same.

$\Leftarrow$ Let $A = M_A H = \delta^{-k} \bar{A}$ and $B = M_B H = \delta^{-k'} \bar{B}$, so that A and B have the same UHNF. Then $M_A^{-1} \delta^{-k} \bar{A} = M_B^{-1} \delta^{-k'} \bar{B}$. Using the same logic as the forwards proof, we have that $k = k' \Rightarrow \bar{A} = M_A M_B^{-1} \bar{B} \Rightarrow A = MB$ for $M = M_A M_B^{-1}$. $\square$

Corollary 2.18. *Two matrices over $\mathbb{Z}[\omega, 1/2]$ have the same UHNF in the Barnes-Wall basis if they only differ by pre-multiplication by a Clifford matrix.*

This follows from Lemma 2.17 and Theorem 2.10.

2.4.2 Smith Normal Form

A full-rank square matrix over $\mathbb{Z}[\omega]$ in Smith Normal Form is a diagonal matrix where each element on the diagonal divides the next. A method to compute the Smith Normal Form of a matrix is described in Algorithm 7 in the appendix.

Theorem 2.19 (Theorem 3.8 in [9]). *Let A be an $N \times N$ matrix on $\mathbb{Z}[\omega]$, then there exist $\mathbb{Z}[\omega]$ -unimodular matrices M_R, M_L such that $A = M_L D M_R$ and D is in Smith Normal Form. The elements on the diagonal of D are the **invariant factors** of A , and are unique up to multiplication by units. [4]*

Corollary 2.20. *Two matrices A, B on $\mathbb{Z}[\omega]$ have the same invariant factors if and only if there exist $\mathbb{Z}[\omega]$ -unimodular matrices M_L, M_R such that $A = M_L B M_R$.*

Proof.

$\Rightarrow$ Let A, B be matrices over $\mathbb{Z}[\omega]$ and assume that $A = M_L B M_R$ where M_L, M_R are $\mathbb{Z}[\omega]$ -unimodular matrices. By Theorem 2.12, $A = M_1 D M_2$ where D is the Smith Normal Form of A . Then, $B = M_L^{-1} M_1 D M_2 M_R^{-1}$ and so B has the same Smith Normal Form, D , as A . The invariant factors of A, B are the elements on the diagonal of D so A, B share invariant factors.

$\Leftarrow$ Let $A = M_L D M_R$ and $B = M'_L D' M'_R$. By assuming that A, B share invariant factors, each element D_{ii}, D'_{ii} on the diagonal are associates, so $D_{ii} = u_i D'_{ii}$ for some u_i a unit in $\mathbb{Z}[\omega]$. We can construct a diagonal matrix M such that $D_{ii} = M_{ii} D'_{ii}$. M is $\mathbb{Z}[\omega]$ -unimodular since the determinant is the product of the diagonal elements which are all units in $\mathbb{Z}[\omega]$. Thus, $D = M D'$ and we have $A = M_L M M'^{-1} B M'_R \Rightarrow A = M_1 B M_2$ for $\mathbb{Z}[\omega]$ -unimodular matrices M_1, M_2 . $\square$

For the following theorems and lemmas, we use the notations:

- $[A]_m^{I,J}$ is the determinant of the $m \times m$ sub-matrix of an $N \times N$ matrix A made of the rows and columns with indices from the subsets I, J of m elements from $[1, \dots, N]$.
- Δ_m^A is the greatest common divisor of the determinants of all $m \times m$ sub-matrices of A , for $m = 1, \dots, N$.

Theorem 2.21 (Special case of Equation 32 of [8]). *Generalised Cauchy-Binet Formula for Square Matrices:*

Let A, B be $N \times N$ matrices, I, J be subsets of $[1, \dots, N]$ with m elements, then

$$[AB]_m^{I,J} = \sum_K [A]_m^{I,K} [B]_m^{K,J} \quad (7)$$

where the sum is over all subsets K of $[1, \dots, N]$ of m elements.

This theorem gives a relation between the minors of the product, AB , and the minors of A and B . We use it to derive a property of the **greatest common divisor** of minors of a product.

Lemma 2.22. *Let A, B be full-rank $N \times N$ matrices. Then $\Delta_i^A \Delta_i^B$ divides Δ_i^{AB} for all $i = 1, \dots, N$.*

Proof. Δ_i^A and Δ_i^B divide $[A]_i^{I,K}$ and $[B]_i^{K,J}$, respectively, for all subsets I, J, K of i elements. Consequently, using Theorem 2.21, $\Delta_i^A \Delta_i^B$ divides $[AB]_i^{I,J}$ for all subsets I, J . This means that $\Delta_i^A \Delta_i^B$ divides Δ_i^{AB} . $\square$

Theorem 2.23 (Special Case of Theorem 3.9 in [9]). *Let A be a full rank $N \times N$ matrix with entries in $\mathbb{Z}[\omega]$. Then the invariant factors d_j of A are equal (up to units of $\mathbb{Z}[\omega]$) to:*

$$d_1 = \Delta_1^A, d_j = \Delta_j^A / \Delta_{j-1}^A \quad (8)$$

Lemma 2.24. *Let $A = \delta^k U$ where k is the least denominator exponent of U and $\det(U) = 1$. The invariant factors of A are powers of δ up to units.*

Proof. Considering $N \times N$ matrices, let $A = \delta^k U = M_L D M_R$ where M_L, M_R are $\mathbb{Z}[\omega]$ -unimodular matrices and D is the Smith Normal Form of A . Then $\det(A) = \prod_{i \in [1, N]} d_i = (\delta^k)^N$ where d_i are the invariant factors of A . Since δ is prime in $\mathbb{Z}[\omega]$ by Lemma 2.5, each d_i must be a power of δ up to units. $\square$

3 Defining the Graph Search

By transforming the synthesis problem into a graph search problem, we can use well-known shortest-path search algorithms to find the T-optimal synthesis of a circuit unitary. Effectively, the "length" of the path is related to the number of T gates in the synthesis. We begin by re-writing the circuit unitary U in a more insightful way and by defining a **generator** set for U , and then we use these definitions to construct our graph.

3.1 Representation of U

Let U be a $2^n \times 2^n$ unitary matrix over $\mathbb{Z}[\omega, 1/2]$ which can be synthesised using gates from the Clifford + T set. Let C be the Clifford set, and G be the costly set. We can write U as a sequence of costly gates $g_i \in G$ separated by sequences c_i of elements from C :

$$U = c_M g_M c_{M-1} g_{M-1} \dots g_1 c_1 g_0 c_0 \quad (9)$$

After pairing each $c_i g_i$ and adjusting $\tilde{c}_i$ to match the inverses, we rewrite U :

$$U = \tilde{c}_{M+1} (\tilde{c}_M g_M \tilde{c}_M^{-1}) (\tilde{c}_{M-1} g_{M-1} \tilde{c}_{M-1}^{-1}) \dots (\tilde{c}_0 g_0 \tilde{c}_0^{-1}) \quad (10)$$

Defining $\tilde{g}_i = \tilde{c}_i g_i \tilde{c}_i^{-1}$ as **generators**:

$$U = \tilde{c}_{M+1} \tilde{g}_M \tilde{g}_{M-1} \dots \tilde{g}_0 \quad (11)$$

This is the form of U that will be referred to in later sections.

3.2 Minimal Generator Sets

By conjugating g with every matrix, c , made of elements from C , the generator set becomes very large. For example, the 2-qubit Clifford group contains 11,520 matrices [13], as would our 2-qubit generator set. Fortunately, we can define a *minimal* generator set by noting the following:

Lemma 3.1. *Let $\tilde{g}_1, \tilde{g}_2 \in \tilde{G}$ be two generators such that $\tilde{g}_1 = c \tilde{g}_2$. Then, the generator $\tilde{g}_1$ is redundant.*

Proof. If the sequence of generators of U , written as in Equation 11, includes $\tilde{g}_1$, we can replace $\tilde{g}_1$ by $c \tilde{g}_2$ and propagate c through the generators to include it in $\tilde{c}_{N+1}$, making the generator $\tilde{g}_1$ redundant. $\square$

We use this to define a minimal set of generators:

Definition 3.2. *The minimal generator set $\tilde{G}$ is the finite set*

$$\tilde{G} = \{c g c^{-1} : c \text{ is a sequence of elements from } C, g \in G\} \quad (12)$$

where $\tilde{g}_i \neq c \tilde{g}_j$ for any $\tilde{g}_i, \tilde{g}_j \in \tilde{G}$, c a sequence of gates from C .

This definition removes redundant generators. Using Corollary 2.18 this definition means that each $\tilde{g} \in \tilde{G}$ has a unique UHNF in the Barnes-Wall basis. The set $\tilde{G}$ can be constructed using the following algorithm:

Algorithm 1 (Construction of Minimal Generator Set).

1. Set G to the costly gate set, C to the cost-free set, and define an empty set $\tilde{G}$ and an empty queue W
2. For each $g \in G$ with a unique UHNF in the Barnes-Wall basis, add g to $\tilde{G}$ and add all combinations of cgc^{-1} where c is a **single** element from C to the back of W .
3. Repeat until W is empty:
 - Remove the front element w from W
 - If the UHNF of w in the Barnes-Wall basis is distinct to all $\tilde{g} \in \tilde{G}$, add w to $\tilde{G}$ and add all combinations of cwc^{-1} where c is a **single** element from C to the back of W .
 - If the UHNF of w is not distinct, discard it.

Using this algorithm, we obtain 3 generators for the 1-qubit Clifford + T set and 15 for the 2-qubit Clifford + T set. These can be found in the appendix.

3.3 Vertices and Edges

When implementing a shortest-path graph search, we begin at a source vertex and traverse the graph to a goal vertex. Vertices are connected by edges with associated costs and a shortest path is one with the lowest total edge cost.

Definition 3.3 (Vertices). *Each vertex is an equivalence class of $2^n \times 2^n$ matrices over $\mathbb{Z}[\omega, 1/2]$ having the same UHNF in the Barnes-Wall basis. Equivalently, the vertex v_A containing the matrix A is defined as:*

$$v_A = \{cA : c \text{ is a sequence of gates from } C\} \quad (13)$$

Using the representation of U in Equation 11, this means that any matrix with the same sequence of generators belongs to the same vertex, as they only differ in $\tilde{c}_{M+1}$.

Definition 3.4 (Source Vertex). *The source vertex is the vertex v_U containing the unitary U representing the circuit.*

Definition 3.5 (Goal Vertex). *The goal vertex is the vertex v_c containing only sequences of matrices from C .*

Definition 3.6 (Edges). *Each edge represents an element of the finite minimal generator set.*

The degree of each vertex is equal to the number of elements in the minimal generator set $\tilde{G}$. Traversing the graph corresponds to post-multiplying U by the inverse of generators until we reach the vertex v_c , at which point we are left with the matrix $\tilde{c}_{M+1}$.

Definition 3.7. *A vertex is considered **expanded** once its neighbours have been calculated. By default, these newly created neighbours are considered **unexpanded**.*

4 A* Search

The Clifford + T gate set is universal, meaning it can approximate any unitary circuit to arbitrary accuracy. As a consequence, the problem graph defined in Section 3 is infinite. We therefore need an efficient algorithm to find the shortest-path from the source vertex to the goal vertex. An A* Search is a well-known guided search algorithm which uses a heuristic function to estimate how promising investigating a particular path might be.

4.1 Heuristics

Definition 4.1 (Heuristics in the A* Algorithm). *A **heuristic function** is a function that evaluates a vertex and returns an estimate of the remaining cost to reach the goal vertex.*

Definition 4.2 (Consistency). *A heuristic is **consistent** if for any neighbouring vertices v, v' connected by an edge with cost g , we have that $h(v) \leq h(v') + g$ and the heuristic value of a goal node is 0.*

Definition 4.3 (Admissibility). *A heuristic is **admissible** if it never overestimates the actual cost to reach a goal vertex: for all vertices v , $h(v) \leq \text{actual cost to reach } v_{\text{goal}}$ from v .*

Lemma 4.4. *A consistent heuristic is admissible.*

Proof. Let h be a consistent heuristic. Then for any two vertices v, v' , we have $h(v) \leq h(v') + g(v, v')$ where $g(v, v')$ is the cost of the edge joining the vertices. Consider the final vertices, v_n, v_{n-1} along a path, so $h(v_{n-1}) \leq h(v_n) + g(v_{n-1}, v_n)$. If v_n is the goal vertex, then $g(v_{n-1}, v_n)$ is the actual cost to reach the goal from vertex v_{n-1} and $h(v_n) = 0$. By iteratively applying the inequality $h(v_{i-1}) \leq h(v_i) + g(v_{i-1}, v_i)$:

$$h(v_i) \leq \sum_{j=i \dots n-1} g(v_j, v_{j+1}) \quad (14)$$

Since the sum on the right-hand side is the actual cost to reach the goal vertex from v_i , $h(v)$ is an admissible heuristic. $\square$

4.1.1 LDE Heuristic

Definition 4.5 (Least Denominator Exponent). *Let V be an $N \times N$ matrix over $\mathbb{Z}[\omega, 1/2]$ and $k \in \mathbb{N}$ be the smallest exponent such that $\delta^k V$ is a matrix with entries in $\mathbb{Z}[\omega]$. We say that k is the **least denominator exponent** of V .*

We now define a consistent heuristic using the least denominator exponent of a matrix at a vertex in the Barnes-Wall basis.

Lemma 4.6 (LDE Heuristic). *Let v_V be a vertex containing the matrix V , and g be a generator representing an edge. Let $V_B = B^{-1}VB$ and $g_B^{-1} = B^{-1}g^{-1}B$. The least denominator exponent of V_B is a consistent heuristic if the edge cost is taken to be the least denominator exponent of g_B^{-1} .*

Proof. Let v_V, v_U be two vertices connected by the generator g such that $U = Vg^{-1}$. Let k_V, k_U , and k_g be the least denominator exponents of V_B, U_B , and g_B^{-1} , respectively. Then since $V_B\delta^{k_V}$ and $g_B^{-1}\delta^{k_g}$ are integer matrices, we have that $V_B g_B \delta^{k_V+k_g} = U_B \delta^{k_U+k_g}$ is also an integer matrix. By definition, k_U is the smallest $k \in \mathbb{N}$ such that $U_B \delta^{k_U}$ is an integer matrix, so it must be $k_U \leq k_V + k_g$. Furthermore, if $h(v_V) = 0$, then the least denominator exponent of V_B is 0, and V_B is an integer matrix over $\mathbb{Z}[\omega]$ with unit determinant. By Theorem 2.10 this is equivalent to V being a Clifford matrix and v_V is a goal vertex. $\square$

The LDE Heuristic is also admissible by the Lemma 4.4, which means that the heuristic value is always an underestimate of the true cost to reach the goal vertex. When calculated, it can be shown that each generator in the Barnes-Wall basis (for 1- and 2- qubit Clifford + T) has a least denominator exponent of 1, so can reduce the heuristic value by at most 1. As such, the heuristic value of the starting vertex $h(v_U)$ gives a lower bound on the optimal T-count.

4.1.2 $\bar{k}$ -Heuristic

Using the Smith Normal Form of matrices over $\mathbb{Z}[\omega]$ we now define a more advanced heuristic.

Lemma 4.7. *Let U be a $N \times N$ unitary matrix over $\mathbb{Z}[\omega, 1/2]$, and B be the $N \times N$ Barnes-Wall basis matrix. There exist $\mathbb{Z}[\omega]$ -unimodular matrices M_L, M_R such that*

$$U_B = B^{-1}UB = M_L \text{diag}(\delta^{k_1}, \dots, \delta^{k_{(N/2)}}, \delta^{-k_{(N/2)}}, \dots, \delta^{-k_1}) M_R \quad (15)$$

where $k_i \in \mathbb{Z}$ and $k_i \geq k_{i+1} \geq 0$ for $i = 1 \dots N/2$. $N/2$ is always an integer as it is equal to 2^n where n is the number of qubits.

Proof. Let $A = \delta^k B^{-1}UB = M_L D M_R$ such that k is the least denominator exponent of $B^{-1}UB$. In the following proof, we use the fact that $U^\dagger U = I$ and that $B^\dagger B$ is δ -modular from Lemma 2.12. Starting with $U^\dagger U = I$ and substituting $U = \delta^{-k} B A B^{-1}$, we obtain

$$(\delta \delta^*)^{-k} (B^{-1})^\dagger A^\dagger B^\dagger (B A B^{-1}) = I \quad (16)$$

By re-ordering, and noting that $\delta^* = \delta u$ for a unit u , which we will not track, this becomes:

$$\delta^{2k} B^\dagger B = A^\dagger B^\dagger B A \quad (17)$$

Using Lemma 2.12, we can replace $B^\dagger B$ by $\delta^{-l} M_B$ where M_B is a $\mathbb{Z}[\omega]$ -unimodular matrix:

$$\delta^{2k} M_B = A^\dagger M_B A \quad (18)$$

Substituting $M_L D M_R$ for A and rearranging:

$$\delta^{2k} D^{-1\dagger} = (M_L^\dagger M_B M_L) D (M_R M_B^{-1} M_R^\dagger) \quad (19)$$

We denote $|x|_\delta$ to be the δ -adic order of x . That is, the power of δ that appears in the prime factorisation of x .

Let d_i be the i -th element on the diagonal of D and d'_i be the i -th element on the diagonal of $D^{-1\dagger}$, so

$$|d'_i|_\delta = -|d_i|_\delta. \quad (20)$$

By Corollary 2.20, $\delta^{2k} D^{-1^\dagger}$ has the same invariant factors as D , and by Lemma 2.24, the invariant factors of D are powers of δ . Since D and $\delta^{2k} D^{-1^\dagger}$ are diagonal and share invariant factors, this means that the elements of $\delta^{2k} D^{-1^\dagger}$ must be the same as those of D , but permuted. It follows that for any d_i' there is a d_j such that $|d_i'|_\delta + 2k = |d_j|_\delta$. Substituting Equation (15), we get that for any d_i , there is a d_j such that:

$$|d_i|_\delta + |d_j|_\delta = 2k \quad (21)$$

From the definition of the Smith Normal Form, $d_i | d_{i+1}$ so $|d_i|_\delta \leq |d_{i+1}|_\delta$ and we conclude that $|d_i|_\delta = 2k - |d_{N-i}|_\delta$. Taking $k_i = |d_i|_\delta$, we rewrite:

$$A = M_L \text{diag}(\delta^{k_1}, \dots, \delta^{k_{N/2}}, \delta^{2k - (k_{N/2})}, \dots, \delta^{2k - (k_1)}) M_R \quad (22)$$

By substituting $A = \delta^k U_B$:

$$U_B = M_L \text{diag}(\delta^{k_1 - k}, \dots, \delta^{k_{N/2} - k}, \delta^{k - (k_{N/2})}, \dots, \delta^{k - (k_1)}) M_R = M_L \text{diag}(\delta^{\tilde{k}_1}, \dots, \delta^{\tilde{k}_{N/2}}, \delta^{-\tilde{k}_{N/2}}, \dots, \delta^{-\tilde{k}_1}) M_R \quad (23)$$

where $0 \leq \tilde{k}_i \leq \tilde{k}_{i+1}$ for $i = 1 \dots N/2$. However, permutations matrices are $\mathbb{Z}[\omega]$ -unimodular which allows us to permute the elements on the diagonal so that the condition $k_i \geq k_{i+1} \geq 0$ is true. $\square$

Definition 4.8. Let U, k_i be as in Lemma 4.7, we define the vector $\bar{k}(U)$ to be $(k_1, \dots, k_{N/2})$.

Definition 4.9. $K(U) = \sum_{i=1}^{N/2} \bar{k}(U)_i$.

Lemma 4.10. $K(c) = 0$ if and only if c is a sequence of elements from C .

Proof.

$\Rightarrow$ Using Lemma 2.10, if c is a sequence of elements from C , then $B^{-1}cB$ is $\mathbb{Z}[\omega]$ -unimodular. The diagonal matrix in Equation 4.7 is therefore the identity matrix up to units and $(k_1, \dots, k_{N/2}) = (0, \dots, 0)$.

$\Leftarrow$ Each $k_i \geq 0$ so $K(c) = 0$ if and only if $k_i = 0$ for all i . The diagonal matrix in Equation 4.7 must therefore be the identity matrix up to units, as Lemma 2.24 requires that every element on the diagonal be a power of δ . This means that the left-hand-side of Equation 4.7 must be equal to a $\mathbb{Z}[\omega]$ -unimodular matrix. Finally, using Lemma 2.10, this means that c must be a sequence of elements from C . $\square$

Lemma 4.11. The first entry in $\bar{k}(U)$ is the lowest denominator exponent of U_B .

Proof. Every invariant factor of A in Equation 22 is a power of δ by Lemma 2.24, and the first invariant factor is also the gcd of all entries in A by Lemma 2.23. Since k is the lowest denominator exponent of U_B , the gcd of A cannot be a positive integer power of δ , therefore we have that $k_1 = 0$ in Equation 22. The first entry in $\bar{K}(U)$ is $k - k_1$ where k is the lowest denominator exponent of U_B . $\square$

Lemma 4.12 ($\bar{k}$ -Heuristic). Let v_V be a vertex containing the matrix V , and g be a generator representing an edge. Then $h(v_V) = K(V)$ is a consistent heuristic if the edge cost is taken to be $K(g^{-1})$.

Proof. Firstly, we note from Lemma 4.10 that $h(v_c) = 0$ if and only if c is a sequence of elements from C . Thus, the heuristic value of a goal vertex is 0. Secondly, we must show that $K(Vg^{-1}) \leq K(V) + K(g^{-1})$ for all vertices v_V and generators $g \in \tilde{G}$.

Let $\tilde{U} = \delta^{k_1} B^{-1}UB$ and $\tilde{V} = \delta^{k_1'} B^{-1}VB$, where k_1, k_1' are the least denominator exponents of $B^{-1}UB$ and $B^{-1}VB$ respectively. Using Lemma 4.7,

$$\tilde{U} = M_L \text{diag}(\delta^{k_1 + k_1}, \dots, \delta^{k_{N/2} + k_1}, \delta^{-k_{N/2} + k_1}, \dots, \delta^{-k_1 + k_1}) M_R \quad (24)$$

The elements are re-ordered such that each element on the diagonal divides the next and so the Smith Normal Form of $\tilde{U}$ is

$$\text{diag}(\delta^{-k_1 + k_1}, \dots, \delta^{-k_{N/2} + k_1}, \delta^{k_{N/2} + k_1}, \dots, \delta^{k_1 + k_1}) \quad (25)$$

Using Theorem 2.23,

$$\Delta_j^{\tilde{U}} = \Pi_i^j d_i = \delta^{\sum_i^j k_i - k_i} \text{ for } j = 1, \dots, N/2 \quad (26)$$

Deriving the same expression for $\tilde{V}$, we then obtain:

$$\Delta_j^{\tilde{U}} \Delta_j^{\tilde{V}} = \delta^{\sum_i^j k_i + k_1' - k_i - k_i'} \text{ for } j = 1, \dots, N/2 \quad (27)$$

Letting $\tilde{U}\tilde{V} = \delta^{k_1 + k_1'} B^{-1}UVB$ and k_i'' denote the exponents relating to $\tilde{U}\tilde{V}$:

$$\text{SNF}(\tilde{U}\tilde{V}) = \text{diag}(\delta^{k_1 + k_1' - k_1''}, \dots, \delta^{k_1 + k_1' - k_{N/2}''}, \delta^{k_1 + k_1' + k_{N/2}''}, \dots, \delta^{k_1 + k_1' + k_1''}) \quad (28)$$

Applying Theorem 2.23 to $\tilde{U}\tilde{V}$:

$$\Delta_j^{\tilde{U}\tilde{V}} = \Pi_i^j d_i = \delta^{\sum_i^j k_i + k'_1 - k''_i} \text{ for } j = 1, \dots, N/2 \quad (29)$$

Applying Lemma 2.22 and expressing it as a sum of the exponents k :

$$\sum_i^j k''_i \leq \sum_i^j k_i + k'_i \text{ for all } j = 1 \dots N/2 \quad (30)$$

When $j = N/2$, this is exactly the statement that $K(Ug^{-1}) \leq K(U) + K(g^{-1})$.

□

Similar to the LDE heuristic, we can use the heuristic value of the starting vertex to give a lower bound on the optimal T-count. The first value in the vector $\bar{k}(U)$ is always the lowest denominator exponent of the matrix, and is therefore a lower bound.

4.2 Algorithm

Definition 4.13. A priority queue is an ordered list where entries are removed from the head, and inserted in order according to an assigned value.

In the A* search in this section, we use a priority queue to store unexpanded vertices according to some value, f , calculated for each vertex, v , using the following formula:

$$f(v) = h(v) + g(v) \quad (31)$$

where h is a consistent heuristic function, and g is the actual cost of the path from the source vertex to the vertex, v . The actual cost is calculated as the sum of the costs of the generators used along the path. At each step we expand the vertex at the head of the priority queue, which is the "most promising" vertex.

Lemma 4.14. An A* search using a consistent heuristic is guaranteed to find the optimal path, if a path exists.

Proof. We use the fact that a consistent heuristic is also admissible, and use the notation that $g(i, j)$ is the cost of the edge connecting the vertices i and j .

Suppose there are two paths to a goal vertex, P, P^* , whose vertices we note as P_j, P^*_j . Let P^* be the optimal path, so $\sum_{start}^{goal} g(P^*_j, P^*_{j+1}) < \sum_{start}^{goal} g(P_j, P_{j+1})$. Suppose the A* Search returns the sub-optimal path P , instead of the optimal path P^* . For the A* Search to reach the goal vertex from the penultimate vertex on path P , we must have that $h(goal) + \sum_{start}^{goal} g(P_j, P_{j+1}) < h(P^*_i) + \sum_{start}^i g(P^*_j, P^*_{j+1})$ for every vertex P^*_i . Noting that the heuristic value of a goal vertex is 0, and that by admissibility, $h(v_i) \leq \sum_{j=i}^{goal} g(v_j, v_{j+1})$: we have $\sum_{start}^{goal} g(P_j, P_{j+1}) < \sum_{start}^{goal} g(P^*_j, P^*_{j+1})$. This defines P as the optimal path, not P^* . Thus the A* Search returns an optimal path. □

The explicit circuit synthesis algorithm using an A* search is as follows:

Algorithm 2 (A* Synthesis Algorithm).

1. Let W be an empty priority queue, f be as defined in (31), v_U be the vertex containing the circuit matrix U , and $\tilde{G}$ be the minimal generator set computed using Algorithm 1.
2. Insert the vertex v_U as the head of W by calculating $h(v_U)$ and setting $g(v_U)$ to 0.
3. Repeat until a goal vertex is found in step c):
 - (a) Remove the vertex v_V , containing the matrix V , from the head of the queue W and expand v_V . For each generator $\tilde{g} \in \tilde{G}$:
 - (b) Calculate the matrix $V\tilde{g}^{-1}$.
 - (c) Evaluate the heuristic value h for the vertex $v_{V\tilde{g}^{-1}}$.
 - (d) Calculate the actual cost g for the vertex $v_{V\tilde{g}^{-1}}$ by adding the cost of the generator $\tilde{g}$ to the cost of the vertex v_V .
 - (e) Place the vertex $v_{V\tilde{g}^{-1}}$ in W according to the function f in Equation 31.

Using the representation of U in Equation 11, the goal vertex will hold the matrix $U\tilde{g}_0^{-1}\tilde{g}_1^{-1}\dots\tilde{g}_M^{-1} = \tilde{c}_{M+1}$. If we also keep track of the sequence of generators used to get to each vertex, at the goal vertex we will obtain the cost-optimal sequence of generators to synthesise U . This makes the A* search a valid solution to Problem 1.

4.3 Demonstration

We use the notation that $WX.YZ = (W \otimes X) * (Y \otimes Z)$. As an example, let us consider the 2-qubit Clifford + T case. Explicitly, the 2 qubit Clifford set, C , is $\{I\tilde{H}, \tilde{H}I, IS, SI, CX\}$, and the costly set, G , is $\{IT, TI\}$, where I is the 2×2 identity matrix, and $\tilde{H}, S, CX, T$ are as defined in Equation 4.

Firstly, we build the minimal generator set using Algorithm 1 which contains 15 generators with a unique UHNF. This list of generators can be found in Section 7.1 of the Appendix.

Consider this example circuit, U , built by tensoring and composing matrices from the gate set:

$$U = CX.IT.\tilde{H}I.SI.CX.TI.SI.\tilde{H}I.IT.CX.SI.IT.CX.\tilde{H}I.TI.I\tilde{H} = \begin{pmatrix} 0 & \frac{1-\omega^2-\omega^3}{(1+\omega)^3} & \frac{-2\omega-3\omega^2-2\omega^3}{(1+\omega)^3} & 0 \\ \frac{-5+5\omega^2+7\omega^3}{(1+\omega)^5} & \frac{-5-7\omega-5\omega^2}{(1+\omega)^5} & \frac{3+2\omega-2\omega^3}{(1+\omega)^5} & \frac{-2\omega-3\omega^2-2\omega^3}{(1+\omega)^5} \\ \frac{5+7\omega+5\omega^2}{(1+\omega)^5} & \frac{5-5\omega^2-7\omega^3}{(1+\omega)^5} & \frac{2\omega+3\omega^2+2\omega^3}{(1+\omega)^5} & \frac{-3-2\omega+2\omega^3}{(1+\omega)^5} \\ \frac{1-\omega^2-\omega^3}{(1+\omega)^3} & 0 & 0 & \frac{-2\omega-3\omega^2-2\omega^3}{(1+\omega)^3} \end{pmatrix} \quad (32)$$

In the Barnes-Wall basis:

$$B^{-1}UB = \begin{pmatrix} \frac{-2\omega-2\omega^2-\omega^3}{(1+\omega)^2} & \frac{-\omega^3}{(1+\omega)} & \frac{-\omega-\omega^2-\omega^3}{(1+\omega)} & 0 \\ \frac{1+2\omega+2\omega^2}{(1+\omega)^2} & \frac{-6-6\omega-2\omega^2+3\omega^3}{(1+\omega)^4} & \frac{\omega}{(1+\omega)} & \frac{-\omega-\omega^2-\omega^3}{(1+\omega)^3} \\ \frac{1+\omega-\omega^3}{(1+\omega)^2} & \frac{-2-3\omega-2\omega^2}{(1+\omega)^4} & \frac{1+\omega+\omega^2}{(1+\omega)} & \frac{-1-\omega+\omega^3}{(1+\omega)^3} \\ \frac{-2-\omega+2\omega^3}{(1+\omega)^2} & -\omega & \frac{-1-\omega+\omega^3}{(1+\omega)} & -\omega \end{pmatrix} \quad (33)$$

Implementing the A* Search using the LDE heuristic, we can see that $h(v_U)$ is 4, which is a lower bound for the optimal T-count. Given we built U by using 5 T gates, we have an upper bound of 5 for the optimal T-count. We therefore expect the algorithm to produce a sequence of either 4 or 5 generators. The *sequence* of generators outputted by the algorithm is:

$$(CX.SI.\tilde{H}I).TI.(CX.SI.\tilde{H}I)^{-1}.(I\tilde{H}).IT.(I\tilde{H})^{-1}.(SI.\tilde{H}I).TI.(SI.\tilde{H}I)^{-1}.(CX.\tilde{H}I).TI.(CX.\tilde{H}I)^{-1}.TI \quad (34)$$

Indeed, this has a T-count of 5. This means that the sequence of tensor products and compositions to build U in Equation 32 was also a T-optimal circuit.

The remaining Clifford matrix outputted by the algorithm, $\tilde{c}_{M+1}$ of Equation 11, is:

$$\tilde{c}_{M+1} = \begin{pmatrix} \frac{3+2\omega-2\omega^3}{(1+\omega)^4} & \frac{-2\omega-3\omega^2-2\omega^3}{(1+\omega)^4} & \frac{-3-2\omega+2\omega^3}{(1+\omega)^4} & \frac{-2\omega-3\omega^2-2\omega^3}{(1+\omega)^4} \\ \frac{3+2\omega-2\omega^3}{(1+\omega)^4} & \frac{2\omega+3\omega^2+2\omega^3}{(1+\omega)^4} & \frac{-3-2\omega+2\omega^3}{(1+\omega)^4} & \frac{2\omega+3\omega^2+2\omega^3}{(1+\omega)^4} \\ \frac{-2\omega-3\omega^2-2\omega^3}{(1+\omega)^4} & \frac{-3-2\omega+2\omega^3}{(1+\omega)^4} & \frac{-2\omega-3\omega^2-2\omega^3}{(1+\omega)^4} & \frac{3+2\omega-2\omega^3}{(1+\omega)^4} \\ \frac{-2\omega-3\omega^2-2\omega^3}{(1+\omega)^4} & \frac{3+2\omega-2\omega^3}{(1+\omega)^4} & \frac{-2\omega-3\omega^2-2\omega^3}{(1+\omega)^4} & \frac{-3-2\omega+2\omega^3}{(1+\omega)^4} \end{pmatrix} \quad (35)$$

We check this is a Clifford matrix by verifying that it is integer in the Barnes-Wall Basis and that the determinant is a unit in $\mathbb{Z}[\omega]$.

$$\tilde{c}_{M+1_B} = \begin{pmatrix} -1 & -1-\omega^2 & -1 & -\omega^2 \\ 1-\omega^2 & 2 & 1 & 1+\omega^2 \\ 0 & 1 & -\omega^2 & 1 \\ \omega^2 & -1+\omega^2 & \omega^2 & -1+\omega^2 \end{pmatrix} \quad \det(\tilde{c}_{M+1_B}) = -1 \quad (36)$$

$\tilde{c}_{M+1_B}$ is $\mathbb{Z}[\omega]$ -unimodular, and therefore $\tilde{c}_{M+1}$ is a Clifford. It can be checked that $\tilde{c}_{M+1} \times \text{sequence} = U$, and so this is a valid T-optimal synthesis of U .

5 Matsumoto-Amano Form Search

1-Qubit Clifford + T circuits have a canonical Matsumoto-Amano form. This forms has useful properties such as being T-optimal and unique.

5.1 Algorithm

The graph problem can be easily adapted to find the Matsumoto-Amano form of a 1-qubit Clifford + T circuit.

Definition 5.1 (Definition 7 of [14]). *The Matsumoto-Amano form of a 1-qubit Clifford + T matrix U is:*

$$U = (I|T)(\tilde{H}T|\tilde{S}\tilde{H}T)C \quad (37)$$

that is, U can be built from a sequence of the form I or T , followed by a sequence from $\{\tilde{H}T, \tilde{S}\tilde{H}T\}$, and then a sequence C of Clifford gates from $\{\tilde{H}, S\}$.

Theorem 5.2 (Theorem 1(I) of [14] & Corollary 3.1 of [6]). *Every 1-qubit Clifford + T matrix U has a unique Matsumoto-Amano form. Furthermore, this form is a T-optimal circuit for U .*

We can adapt the graph-search by redefining our vertices to be equivalence classes of matrices which differ by **post-multiplication** by a Clifford matrix and using a **column-wise** Hermite Normal Form to define unique vertices. The algorithm for this is Algorithm 5 in the appendix. This means that for any $\mathbb{Z}[\omega, 1/2]$ matrices A, B sharing a UNHF, there exists a $\mathbb{Z}[\omega]$ -unimodular matrix M such that $A = BM$. We then move from one vertex to the next by **pre-multiplying** by the inverse of generators, which we take to be $\{\tilde{H}.T, S.\tilde{H}.T\}$.

We use an A* Search using the LDE heuristic, where we begin with two source vertices: $v_{T^{-1}U}, v_U$.

Algorithm 3 (Matsumoto-Amano A* Search).

1. Let W be an empty priority queue, f be as defined in (31), v_U be the vertex containing the circuit matrix U , $\tilde{G}$ be the generator set $\{\tilde{H}.T, S.\tilde{H}.T\}$, and $h(v)$ be the LDE heuristic.
2. Insert the vertices v_U and $v_{T^{-1}U}$ into W by calculating f : $f(v_U) = h(v_U)$, and $f(v_{T^{-1}U}) = h(v_{T^{-1}U}) + \text{the cost of generator } T$.
3. Repeat until a goal vertex is found with heuristic = 0 in step 3(c):
 - Remove the vertex v_V from the head of the queue W and expand v_V : for each generator $\tilde{g} \in \tilde{G}$,
 - Calculate the matrix $\tilde{g}^{-1}V$
 - Evaluate the heuristic value h for the vertex $v_{\tilde{g}^{-1}V}$
 - Calculate the actual cost g for the vertex $v_{\tilde{g}^{-1}V}$ by adding the cost of the generator $\tilde{g}$ to the cost of the vertex v_V
 - Place the vertex $v_{\tilde{g}^{-1}V}$ in W according to the function f in Equation 31

When this algorithm terminates, the goal vertex will be v_C where C is the final sequence of Clifford gates in Equation 37.

5.2 Demonstration

We create an example 1-qubit Clifford + T circuit, U , to synthesise, by multiplying gates from the 1-qubit Clifford + T set:

$$U = T * S * \tilde{H} * T * \tilde{H} * S * T * \tilde{H} * T * S * \tilde{H} * T * \tilde{H} * T = \begin{pmatrix} \frac{17+5\omega-10\omega^2-19\omega^3}{(1+\omega)^6} & \frac{-2+2\omega^2+3\omega^3}{(1+\omega)^6} \\ \frac{-2-3\omega-2\omega^2}{(1+\omega)^6} & \frac{-17-19\omega-10\omega^2+5\omega^3}{(1+\omega)^6} \end{pmatrix} \quad (38)$$

In the Barnes-Wall basis:

$$U_B = \begin{pmatrix} \frac{6-\omega-7\omega^2-9\omega^3}{(1+\omega)^5} & \frac{-1+\omega^2+\omega^3}{(1+\omega)^4} \\ \frac{-10-3\omega+15\omega^2+24\omega^3}{(1+\omega)^6} & \frac{-10-9\omega-3\omega^2+5\omega^3}{(1+\omega)^5} \end{pmatrix} \quad (39)$$

The LDE of U_B is 6 which gives a lower bound of 6 for the optimal T-count, and as we initially built U using 6 T gates, we also have an upper bound of 6. The Matsumoto-Amano form of U should therefore have 6 T gates. Algorithm 3 computes the Matsumoto-Amano form of U to be

$$\text{sequence} = T * \tilde{S}\tilde{H}T * \tilde{H}T * \tilde{S}\tilde{H}T * \tilde{S}\tilde{H}T * \tilde{H}T \quad (40)$$

which indeed uses exactly 6 T gates. The remaining Clifford C is

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad (41)$$

which means we do not even require a final Clifford sequence.

6 Concluding Remarks

In this report, the synthesis algorithms using an A* search developed by V. Kliuchnikov and S. Schönnenbeck in [2] have been detailed and algorithms explicitly stated for 1- and 2- qubit Clifford + T circuits. These algorithms find the cost-optimal sequence of generators to synthesise a given unitary, up to multiplication by a remaining Clifford matrix which is considered cost-free. The algorithm stated finds the Clifford matrix which **pre-multiplies** a sequence of generators by using a **row-wise** Hermite Normal Form to calculate the minimal generator set. We could instead find the circuit synthesis up to a cost-free Clifford which **post-multiplies** the sequence of generators (as in Section 5), by considering the **column-wise** Hermite Normal Form.

The heuristic functions can also be used to provide a lower bound on the optimal T-count of a circuit given that a consistent heuristic never overestimates the cost. It is important to emphasise that in general, if given a circuit unitary, we do not know

by definition any sequence of gates used in its synthesis, so an upper bound cannot be trivially found in the manner used in this report.

The proofs in this report are generally independent of the number of qubits in the circuit. However, the number of generators increases considerably with the number of qubits; the Clifford + T gate set has 3, 15, and 270 elements for 1, 2 and 3 qubits, respectively (Table 7.2 of [2]). Thus, this algorithm is likely to be impractical for synthesising circuits with a large number of qubits.

7 Appendix

7.1 Minimal Generating Sets

Minimal Generator Sets	
1 Qubit Clifford + T	2 Qubit Clifford + T
T $\tilde{H}.T.\tilde{H}^{-1}$ $S.\tilde{H}.T.(S.\tilde{H})^{-1}$	IT $I\tilde{H}.IT.(I\tilde{H})^{-1}$ $IS.I\tilde{H}.IT.(IS.I\tilde{H})^{-1}$ TI $\tilde{H}I.TI.(\tilde{H}I)^{-1}$ $SI.\tilde{H}I.TI.(SI.\tilde{H}I)^{-1}$ $CX.IT.(CX)^{-1}$ $I\tilde{H}.CX.IT.(I\tilde{H}.CX)^{-1}$ $\tilde{H}I.CX.IT.(\tilde{H}I.CX)^{-1}$ $CX.\tilde{H}I.TI.(CX.\tilde{H}I)^{-1}$ $CX.IS.I\tilde{H}.IT.(CX.IS.I\tilde{H})^{-1}$ $SI.\tilde{H}I.CX.IT.(SI.\tilde{H}I.CX)^{-1}$ $CX.\tilde{H}I.CX.IT.(CX.\tilde{H}I.CX)^{-1}$ $CX.SI.\tilde{H}I.TI.(CX.SI.\tilde{H}I)^{-1}$ $IS.CX.\tilde{H}I.TI.(IS.CX.\tilde{H}I)^{-1}$

7.2 Algorithms for Matrix Normal Forms

In these algorithms we use the Extended Euclidean Algorithm:

Theorem 7.1 (Extended Euclidean Algorithm). *In $\mathbb{Z}[\omega]$ there exists an algorithm which takes two values (x,y) and calculates their integer **Bezout coefficients** (a,b) , and their **greatest common divisor** such that $ax + by = \gcd(x,y)$.*

This algorithm exists because $\mathbb{Z}[\omega]$ is a Euclidean Domain. A Euclidean Algorithm over $\mathbb{Z}[\omega]$ is defined in [5] and a basic definition of the Extended Euclidean Algorithm is given in Chapter 4 of [11].

Corollary 7.2 (Bezout Swaps). *There exists a $\mathbb{Z}[\omega]$ -unimodular matrix that transforms (by pre- or post-multiplication) a matrix U into $\tilde{U}$ such that $\tilde{U}_{ij} = \gcd(U_{ij}, U_{kl})$ and $\tilde{U}_{kl} = 0$, where $i = k$ or $j = l$.*

Proof. We demonstrate this using a 2×2 matrix: Suppose we want to transform the vector $\begin{pmatrix} x \\ y \end{pmatrix}$ such that the top entry is $\gcd(x,y)$ and the bottom entry is 0. We can use the Extended Euclidean algorithm to compute Bezout coefficients a, b such that $ax + by = \gcd(x, y)$. The matrix $M = \begin{pmatrix} a & b \\ -y & x \end{pmatrix}$ performs this transformation and is $\mathbb{Z}[\omega]$ -unimodular since $\det(M) = (ax + by)/\gcd = 1$, and each entry is integer. To apply this transformation to matrices we position the coefficients in the appropriate places, as in the example below, where we are using a Bezout Swap to zero out U_{21} :

$$\begin{pmatrix} 2 & -3 & 0 & 0 \\ -5 & 8 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} 24 & 12 & 6 & 5 \\ 15 & 30 & 45 & 12 \\ 14 & 6 & 2 & 6 \\ 20 & 8 & 4 & 5 \end{pmatrix} = \begin{pmatrix} 3 & -66 & -123 & -26 \\ 0 & 180 & 330 & 71 \\ 14 & 6 & 2 & 6 \\ 20 & 8 & 4 & 5 \end{pmatrix} \quad (42)$$

□

To transform $\mathbb{Z}[\omega]$ matrices into Smith and Hermite Normal Forms, we pre- and post- multiply by $\mathbb{Z}[\omega]$ -unimodular matrices. Such matrices include:

- Row/ column additions: For $i \neq j$, add $\lambda \times$ row/column i to row/ column j
- Row/ column swaps: Swap rows/ columns i and j

- Row/ column unit multiplication: Multiply a row/ column i by a unit of $\mathbb{Z}[\omega]$
- Row/column Bezout swaps

Algorithm 4 ((Row-Wise) Hermite Normal Form).

Method to find an HNF of a full-rank square matrix, U . We iterate over the columns, j , from left to right:

1. Set the pivot element to be the element on the diagonal, U_{jj} . If this is zero, swap rows with a row below.
2. For each row, i , below the diagonal use Bezout Swaps to make the pivot be $\gcd(U_{jj}, U_{ij})$ and zero out U_{ij} .
3. For each row, i , above the diagonal use row additions to set U_{ij} to be the residue of U_{ij} modulo U_{jj} .

We conclude with a matrix that has been transformed by $\mathbb{Z}[\omega]$ -unimodular operations which is upper triangular and the norm of the diagonal element in each column is greater than the elements above it.

Algorithm 5 ((Column-Wise) Hermite Normal Form).

Method to find an HNF of a full-rank square matrix, U . We iterate over the rows, j , from bottom to top:

1. Set the pivot element to be the element on the diagonal, U_{jj} . If this is zero, swap columns with a column to the left.
2. For each column, i , to the right of the diagonal, moving from right to left, use Bezout Swaps to make the pivot be $\gcd(U_{jj}, U_{ji})$ and zero out U_{ji} .
3. For each column, i , to the left of the diagonal use column additions to set U_{ji} to be the residue of U_{ji} modulo U_{jj} .

We conclude with a matrix that has been transformed by $\mathbb{Z}[\omega]$ -unimodular operations which is lower triangular and the norm of the diagonal element in each row is greater than the elements to the left of it.

Algorithm 6 (Checking Uniqueness of Hermite Normal Forms).

We attempt to transform one HNF, U , into another, V by using $\mathbb{Z}[\omega]$ -unimodular row operations. If this is possible, they share UHNFs. If the algorithm fails to transform U into the V , they do not. The triangular structure of U and V simplifies the process, and is the reason for pre-computing HNFs.

We iterate over the columns, j , to transform U into V :

1. Verify that $N(U_{jj}) = N(V_{jj})$. The diagonal elements must only differ by a unit, since there are no linear row operations we can do to change the element on the diagonal without disrupting the triangular form, except multiplication by a unit.
2. Multiply row j of U by V_{jj}/U_{jj} . Now $U_{jj} = V_{jj}$
3. We now iterate over the rows above the diagonal, i :
 - Verify that U_{jj} divides $(U_{ij} - V_{ij})$. If it is possible to transform U_{ij} into V_{ij} by linear row operations, then $U_{ij} - V_{ij}$ must be a multiple of U_{jj} . We can only consider multiples of the pivot, as using other rows would disrupt the triangular structure, or columns previously manipulated.
 - Set $a = (U_{ij} - V_{ij})/U_{jj}$ and subtract $a \times$ row j from row i of U
 - Verify that $U_{ij} = V_{ij}$
4. The column j of U is identical to column j of V . Move to the next column.

If we have successfully transformed U into V using $\mathbb{Z}[\omega]$ -unimodular row operations then $MU=V$ and they share a UHNF.

Algorithm 7 (Smith Normal Form).

We first iterate down the diagonal, U_{jj} to transform U into Smith Normal Form:

1. If $U_{jj} = 0$, search down the columns, (and next rows if necessary) to find a non-zero U_{kl} . Use row and column swaps to make $U_{jj} = U_{kl}$
2. We iterate over all rows $i \neq j$:
 - (a) Use Bezout Swaps to make the pivot be $\gcd(U_{jj}, U_{ij})$
3. We iterate over all columns $i \neq j$:
 - (a) Use Bezout Swaps to make the pivot be $\gcd(U_{jj}, U_{ji})$
4. By performing Bezout Swaps over columns in Step 3, U_{jj} may no longer be the gcd of all the elements in the column. Repeat Steps 2 and 3 until U_{jj} is the gcd of all elements in row and column j .
5. Apply row and column additions to zero out the elements in row and column j using U_{jj}
6. Row and column j are in the correct form. Move on to the next diagonal element and repeat

U is now a diagonal matrix and we iterate again down the diagonal, U_{jj} :

1. If U_{jj} divides $U_{j+1,j+1}$, move onto the next element on the diagonal
2. If not, calculate Bezout coefficients, and use row and column operations to make U_{jj} be $\gcd(U_{jj}, U_{j+1,j+1})$. Then U_{jj} divides $U_{j+1,j+1}$ and we move on to the next element on the diagonal

We end with a diagonal matrix where each element on the diagonal divides the next, and we have used only $\mathbb{Z}[\omega]$ -unimodular operations to do so.

7.3 Python Implementations

The GitHub repository can be found at <https://github.com/kkjeiller-dotcom/Graph-Search-Circuit-Synthesis-PYTHON.git>

8 References

References

- [1] M. Amy, A. Glaudell & N. Ross (2019). "Number-Theoretic Characterizations of Some Restricted Clifford + T Circuits" doi.org/10.22331/q-2020-04-06-252
- [2] V. Kliuchnikov & S. Schönnenbeck, (2024). "Multi-qubit Circuit Synthesis and Hermitian Lattices". doi.org/10.48550/arXiv.2405.19302.
- [3] V. Kliuchnikov, & S. Schönnenbeck, (2024). "Stabilizer Operators and Barnes-Wall Lattices". doi.org/10.48550/arXiv.2404.17677.
- [4] P. Conti, (1990). "Hermite Canonical Form and Smith Canonical Form of a Matrix Over a Principal Ideal Domain". doi.org/10.1145/101104.101105
- [5] N. Kaiblinger, (2011). "Cyclotomic Rings with Simple Euclidean QAlgorithm". JP Journal of Algebra, Number Theory and Applications. 23.
- [6] P. Selinger, B. Giles, (2019). "Remarks on Matsumoto and Amano's normal form". arxiv.org/abs/1312.6584 for single-qubit Clifford+T operators
- [7] B. Broxson, (2006). "The Kronecker Product" UNF Graduate Theses and Dissertations. 25. <https://digitalcommons.unf.edu/etd/25>.
- [8] F. Gantmacher, (1959). "The Theory of Matrices Volume 1" Chapter 1.4, AMS Chelsea Publishing
- [9] N. Jacobson, (2009). "Basic Algebra I: Second Edition. Basic Algebra". Dover Publications, isbn: 9780486471891.
- [10] B. Giles & P. Selinger, (2013). "Exact synthesis of multiqubit Clifford+T circuits". doi.org/10.48550/arXiv.1212.0506
- [11] V. Shoup, (2005). "A Computational Introduction to Number Theory and Algebra". Cambridge: Cambridge University Press.
- [12] H. Lenstra, (1975). "Euclid's Algorithm in Cyclotomic Fields". Journal of the London Mathematical Society
- [13] M. Ozols, (2008). "Clifford group". Essays at University of Waterloo, Spring.
- [14] K. Matsumoto & K. Amano, (2008). "Representation of quantum circuits with Clifford and $\pi/8$ gates". [arXiv:0806.3834](https://arxiv.org/abs/0806.3834)